

LABORATORY OBSERVATION / RECORD

Course	Full Stack Web Development
Course Code	23CM4121
Branch	CSM
Section	B
Task No.	5 (PBL - Part 1)
Student Name	P. Govindamma
Roll No.	A24126552104

This record contains four experiments carried out under Task 5. Each experiment is documented with its aim, objective, theory, algorithm, program, code explanation, test cases with valid and invalid inputs, output screenshots and result.

Q. No.	Question	CO
1	Interactive Product Calculator: Create a webpage containing a product name, quantity and price. Using JavaScript functions and event listeners, calculate the total price dynamically whenever the quantity or price changes. Display the result without refreshing the page.	CO-1
2	Node.js File Management: Create a Node.js application that accepts a filename and content from the user and performs create/write, read, append and display of the final contents using the File System module.	CO-2
3	Express.js Student Server: Create a basic Express.js server with routes /, /students and /about. The /students route should return a list of at least five students, while /about should display information about the application.	CO-2
4	Explain the Document Object Model (DOM). Demonstrate how JavaScript can dynamically change the content, style and attributes of an HTML element.	CO-1

EXPERIMENT 1

1. TITLE

Interactive Product Calculator Using JavaScript, Event Listeners and Real-Time DOM Updates.

2. AIM

To build a webpage that takes a product name, quantity and price, and to calculate and display the total price dynamically using JavaScript functions and event listeners whenever the quantity, price or product name changes, without refreshing the page.

3. OBJECTIVE

- To design an input form using HTML and CSS with fields for product name, quantity and price, styled as a card with labelled inputs and dedicated error and result areas.
- To select the input and output elements from JavaScript using `document.getElementById()`.
- To read the values typed by the user with the `.value` property and convert the numeric fields with `parseFloat()`.
- To write a single reusable function `calculateTotal()` that validates the inputs, performs the multiplication and updates the page.
- To attach `calculateTotal()` to the `input` event of the product name, quantity and price fields using `addEventListener()`, so the total recalculates as the user types.
- To validate quantity and price independently, showing a separate error message under each invalid field instead of a single generic message.
- To fall back to a default product name of "Product" when the name field is left blank or contains only spaces, using `.trim()`.
- To update the page through the DOM using the `textContent` property, so the result appears without a page reload, and to format the amount to two decimal places with `toFixed(2)`.

4. THEORY

This program is event-driven. The HTML page is loaded once, and after that every keystroke in the product name, quantity or price field fires an `input` event that runs `calculateTotal()`. Because the result and the error messages are written back into the page through the DOM, the browser never reloads the document, and the total appears to update instantly as the user types.

All three inputs are wired to the same handler function, so the calculation logic exists in exactly one place and stays consistent no matter which field the user is editing. The values returned by the `.value` property are always strings, even for a `type="number"` input, so `parseFloat()` is used to convert the quantity and price into numbers before they are multiplied. Unlike `Number()`, `parseFloat()` returns `NaN` for an empty or non-numeric string rather than silently treating it as zero, which is what makes the `isNaN()` checks in the validation block meaningful.

The key methods and concepts used in this program are:

- `document.getElementById("id")` returns the element on the page whose `id` attribute matches, so JavaScript can read from it or write into it.
- `element.value` reads the current text inside an input field at the moment the function runs.

- `parseFloat(value)` converts the string read from an input into a floating-point number, returning NaN when the string is empty or not a valid number.
- `addEventListener("input", calculateTotal)` runs `calculateTotal()` on every keystroke or spinner click in that field.
- `element.textContent` writes plain text into an element, replacing whatever was there before; this is how the result and the two error messages are displayed.
- `isNaN(quantity) || quantity < 0` rejects a quantity that is missing, non-numeric or negative; the same pattern is applied separately to the price.
- `toFixed(2)` rounds the total to two decimal places and returns it as a string, which is suitable for displaying currency.
- `productNameInput.value.trim() || "Product"` falls back to the word "Product" whenever the name field is empty or contains only whitespace, so the result line is never left without a label.

The basic flow of the page is:

The user types or changes the product name, the quantity or the price → the `input` event fires → `calculateTotal()` runs → both error messages are cleared → the quantity and the price are read and converted with `parseFloat()` → each is validated independently → if either is invalid, the matching error message is shown and the result is reset to "Total: ₹0.00" → if both are valid, the quantity is multiplied by the price and the result is written into the result box, prefixed with the trimmed product name (or "Product" if it was left blank), formatted to two decimal places.

5. ALGORITHM / PROCEDURE

- Create a single file, `index.html`, containing the markup, the CSS and the JavaScript together.
- Add a card container with a heading, a text input for the product name, a number input for the quantity and a number input for the price, each preceded by a label.
- Below the quantity and price inputs, add an empty `div` (`id="quantityError"` / `id="priceError"`) for each field's own validation message.
- Add a result `div`, `id="result"`, initially showing "Total: ₹0.00".
- Style the card, labels, inputs, result box and error text with CSS, keeping the error text red and the result box shown in a light green success colour.
- In the script, select the product name, quantity, price, result and the two error elements using `document.getElementById()`.
- Define a function `calculateTotal()` that reads `quantity.value` and `price.value` and converts both with `parseFloat()`.
- Inside the function, clear both error messages first, then check the quantity and the price separately: each is invalid if it is NaN or negative.
- If either is invalid, set the corresponding error text, reset the result to "Total: ₹0.00" and return from the function.
- If both are valid, multiply the quantity by the price, read the trimmed product name (defaulting to "Product" when blank), and write "`<name> Total: ₹`" + `total.toFixed(2)` into the result `div`.
- Attach `calculateTotal()` to the `input` event of the product name field, the quantity field and the price field using `addEventListener()`.

- Call `calculateTotal()` once immediately after the listeners are attached, so the result reflects the default values as soon as the page loads.
- Save the file, open `index.html` in a browser and test it with valid values, an empty product name, and zero, negative and empty values for quantity and price.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Interactive Product Calculator</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background: #f4f6f8;
    display: flex;
    justify-content: center;
    padding-top: 60px;
  }
  .card {
    background: #ffffff;
    padding: 30px 35px;
    border-radius: 10px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    width: 340px;
  }
  h2 {
    margin-top: 0;
    color: #2c3e50;
    text-align: center;
  }
  label {
    display: block;
    margin-top: 14px;
    font-weight: bold;
    color: #333;
  }
  input {
    width: 100%;
    padding: 8px;
    margin-top: 6px;
    border: 1px solid #ccc;
    border-radius: 5px;
    box-sizing: border-box;
    font-size: 15px;
  }
  #result {
    margin-top: 22px;
    padding: 15px;
    background: #eafbea;
    border: 1px solid #b7e4b7;
    border-radius: 6px;
    font-size: 18px;
    font-weight: bold;
    color: #256029;
    text-align: center;
  }
  .error {
    color: #b00020;
    font-size: 13px;
    margin-top: 4px;
    min-height: 16px;
  }
</style>
</head>
<body>

<div class="card">
  <h2>Product Calculator</h2>

  <label for="productName">Product Name</label>
  <input type="text" id="productName" placeholder="e.g. Notebook" value="Notebook">

  <label for="quantity">Quantity</label>
  <input type="number" id="quantity" placeholder="e.g. 3" value="3" min="0">
  <div class="error" id="quantityError"></div>
</div>
```

```

<label for="price">Price per unit (₹)</label>
<input type="number" id="price" placeholder="e.g. 50" value="50" min="0">
<div class="error" id="priceError"></div>

<div id="result">Total: ₹0.00</div>
</div>

<script>
// Grab references to the DOM elements we need
const productNameInput = document.getElementById('productName');
const quantityInput    = document.getElementById('quantity');
const priceInput        = document.getElementById('price');
const resultDiv         = document.getElementById('result');
const quantityError     = document.getElementById('quantityError');
const priceError        = document.getElementById('priceError');

/**
 * calculateTotal()
 * Reads the current quantity and price, validates them,
 * and updates the result div dynamically (no page reload).
 */
function calculateTotal() {
  const quantity = parseFloat(quantityInput.value);
  const price = parseFloat(priceInput.value);

  // Reset previous error messages
  quantityError.textContent = "";
  priceError.textContent = "";

  let hasError = false;

  // Validation
  if (isNaN(quantity) || quantity < 0) {
    quantityError.textContent = "Enter a valid quantity (0 or more).";
    hasError = true;
  }
  if (isNaN(price) || price < 0) {
    priceError.textContent = "Enter a valid price (0 or more).";
    hasError = true;
  }

  if (hasError) {
    resultDiv.textContent = "Total: ₹0.00";
    return;
  }

  const total = quantity * price;
  const name = productNameInput.value.trim() || "Product";

  // Update the DOM with the calculated total
  resultDiv.textContent = `${name} Total: ₹${total.toFixed(2)}`;
}

// Event listeners: recalculate whenever quantity or price changes
quantityInput.addEventListener('input', calculateTotal);
priceInput.addEventListener('input', calculateTotal);
productNameInput.addEventListener('input', calculateTotal);

// Run once on page load so the initial values are reflected
calculateTotal();
</script>
</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById()</code>	Selects the product name, quantity, price, result and the two error elements from the page by their id attribute.
<code>element.value</code>	Reads the text currently typed into an input field as a string.
<code>parseFloat()</code>	Converts the string read from a numeric input field into a floating-point number, giving NaN for an empty or invalid string.

Code	Explanation
<code>addEventListener("input", calculateTotal)</code>	Runs <code>calculateTotal()</code> every time the user types in or changes the product name, quantity or price field.
<code>quantityError.textContent = "" / priceError.textContent = ""</code>	Clears both error messages at the start of every calculation, so a fixed error does not linger on screen.
<code>isNaN(quantity) quantity < 0</code>	Validation condition, applied separately to quantity and price, that rejects a missing, non-numeric or negative value.
hasError flag with early return	Tracks whether either field failed validation; when set, the result is reset and the function exits before any multiplication happens.
<code>quantity * price</code>	Calculates the total price once both validated numbers are available.
<code>productNameInput.value.trim() "Product"</code>	Uses the trimmed product name, or the fallback word "Product" when the field is empty or only spaces.
<code>toFixed(2)</code>	Rounds the total to two decimal places so the amount is displayed in currency format.
<code>resultDiv.textContent</code>	Writes the calculated total, prefixed with the product name, back into the page through the DOM, without reloading it.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and the fields were edited using Playwright-driven keystrokes so that every input event fired exactly as it would for a person typing. The values below were read back from the live DOM after each change, not typed in by hand into this record.

Test Case	Action / Input	Expected Result	Actual Result
TC-01	Open index.html (defaults: Notebook, qty 3, price 50)	Result shows "Notebook Total: ₹150.00" with no errors	Notebook Total: ₹150.00 — Pass
TC-02	Name: Wireless Mouse, Quantity: 2, Price: 750	Total updates to ₹1500.00 without reloading	Wireless Mouse Total: ₹1500.00 — Pass
TC-03	Change the quantity from 2 to 5 (price unchanged)	Total recalculates immediately to ₹3750.00	Wireless Mouse Total: ₹3750.00 — Pass
TC-04	Clear the price field (invalid: empty)	"Enter a valid price (0 or more)." is shown and the total resets to ₹0.00	Price error shown, total ₹0.00 — Pass
TC-05	Restore price to 750, clear quantity (invalid: empty)	"Enter a valid quantity (0 or more)." is shown and the total resets to ₹0.00	Quantity error shown, total ₹0.00 — Pass
TC-06	Quantity: -3, Price: 750 (invalid: negative quantity)	Quantity error is shown and no total is calculated	Quantity error shown, total ₹0.00 — Pass
TC-07	Quantity: 2, Price: -50 (invalid: negative price)	Price error is shown and no total is calculated	Price error shown, total ₹0.00 — Pass
TC-08	Quantity: -2, Price: empty (both invalid at once)	Both the quantity error and the price error are shown together	Both errors shown, total ₹0.00 — Pass
TC-09	Quantity: 0, Price: 750 (boundary: zero quantity)	0 is treated as valid (not negative), so the total is ₹0.00 with no error	No error shown, total ₹0.00 — Pass

Test Case	Action / Input	Expected Result	Actual Result
TC-10	Name: USB Cable, Quantity: 3, Price: 99.99	Total is ₹299.97, rounded to two decimal places	USB Cable Total: ₹299.97 — Pass
TC-11	Product name left as spaces only, Quantity: 3, Price: 120	Blank/whitespace name falls back to the label "Product"	Product Total: ₹360.00 — Pass
TC-12	Product name restored to USB Cable; price cleared then re-entered as 120	Price error clears the moment a valid price is typed again	USB Cable Total: ₹360.00, error cleared — Pass

Note on TC-09: because the validation condition checks `quantity < 0` rather than `quantity <= 0`, a quantity of exactly zero is accepted as valid and produces a total of ₹0.00 with no error message, which differs from a stricter "quantity must be greater than zero" rule.

9. OUTPUT

TC-01 – Page loaded with default values (Notebook, qty 3, price 50)

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Quantity

Price per unit (₹)

Notebook Total: ₹150.00

TC-02 – Valid name, quantity and price entered

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Wireless Mouse

Quantity

2

Price per unit (₹)

750

Wireless Mouse Total: ₹1500.00

TC-03 – Quantity changed from 2 to 5, total recalculated without reload

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Wireless Mouse

Quantity

5

Price per unit (₹)

750

Wireless Mouse Total: ₹3750.00

TC-04 – Invalid input: price field left empty

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Quantity

Price per unit (₹)

Enter a valid price (0 or more).

Total: ₹0.00

TC-05 – Invalid input: quantity field left empty

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Wireless Mouse

Quantity

e.g. 3

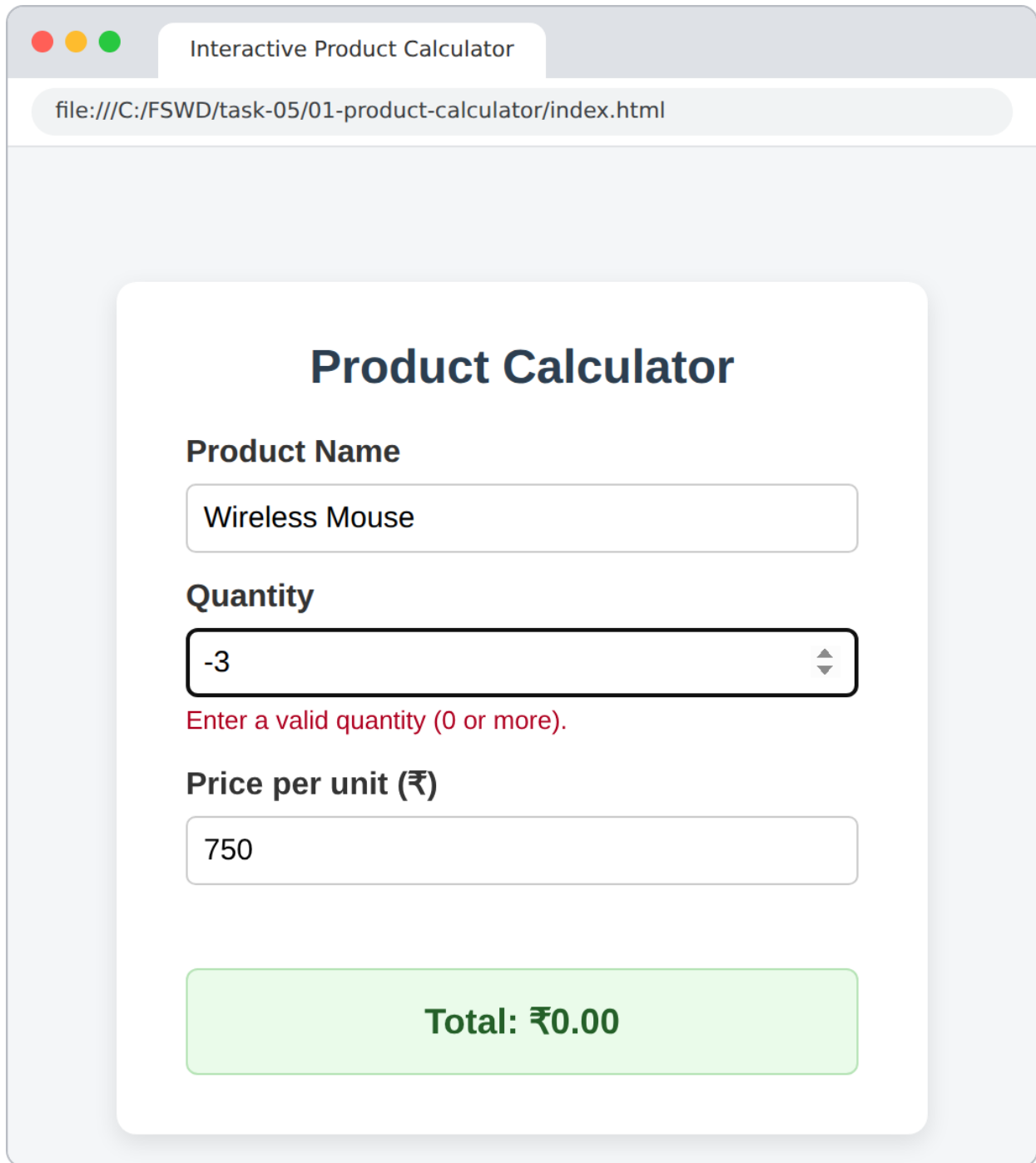
Enter a valid quantity (0 or more).

Price per unit (₹)

750

Total: ₹0.00

TC-06 – Invalid input: negative quantity



The screenshot shows a web browser window with the title 'Interactive Product Calculator'. The address bar displays the file path: `file:///C:/FSWD/task-05/01-product-calculator/index.html`. The main content area features a 'Product Calculator' form. The form has three input fields: 'Product Name' (containing 'Wireless Mouse'), 'Quantity' (containing '-3'), and 'Price per unit (₹)' (containing '750'). Below the 'Quantity' field, a red error message reads: 'Enter a valid quantity (0 or more)'. At the bottom of the form, a green box displays the total: 'Total: ₹0.00'.

Product Calculator

Product Name

Wireless Mouse

Quantity

-3

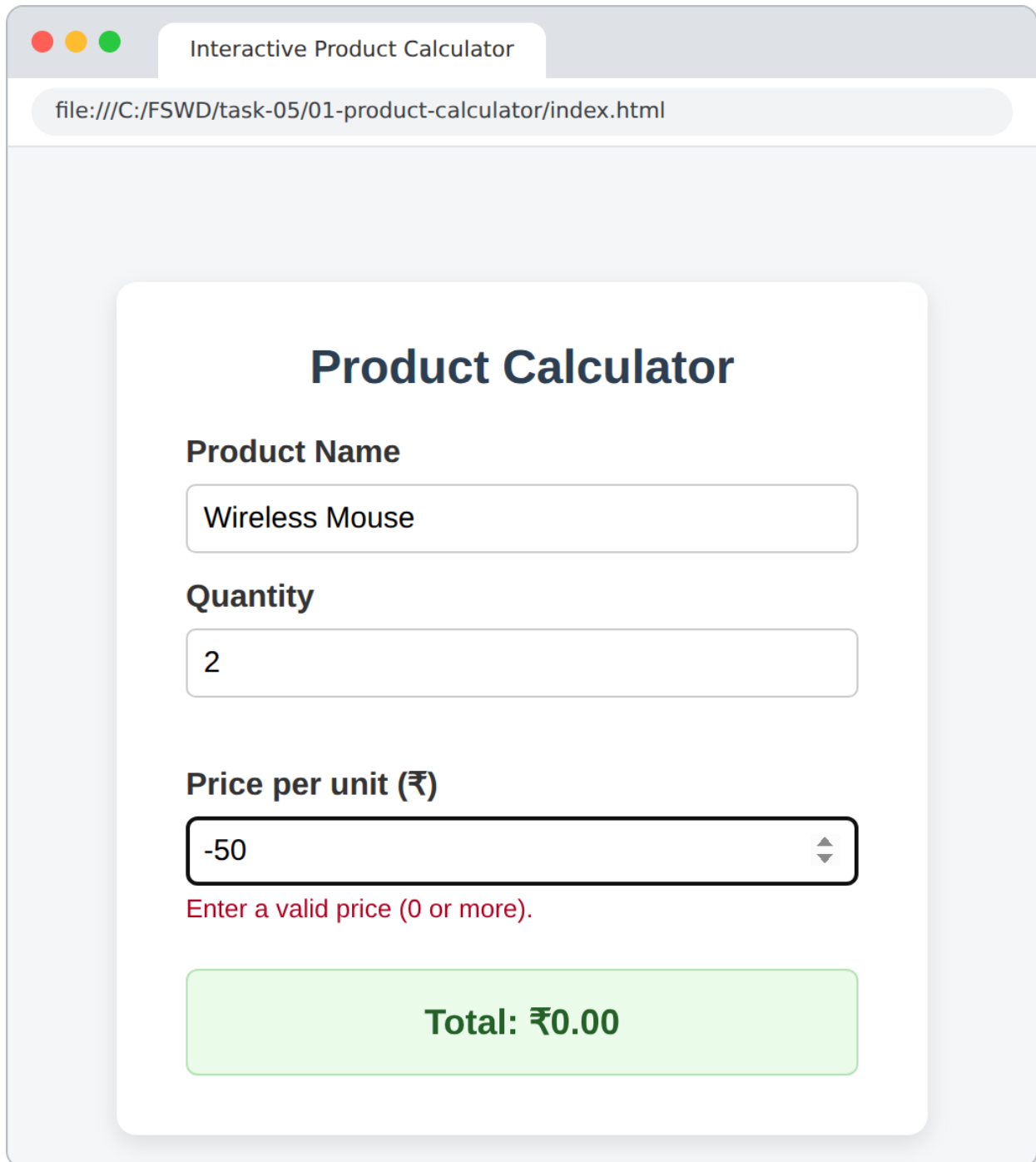
Enter a valid quantity (0 or more).

Price per unit (₹)

750

Total: ₹0.00

TC-07 – Invalid input: negative price



The screenshot shows a web browser window with the title 'Interactive Product Calculator'. The address bar displays the file path: `file:///C:/FSWD/task-05/01-product-calculator/index.html`. The main content area features a 'Product Calculator' form. The form has three input fields: 'Product Name' with the value 'Wireless Mouse', 'Quantity' with the value '2', and 'Price per unit (₹)' with the value '-50'. Below the price input field, there is a red error message: 'Enter a valid price (0 or more)'. At the bottom of the form, a green box displays the total: 'Total: ₹0.00'.

Product Calculator

Product Name

Wireless Mouse

Quantity

2

Price per unit (₹)

-50

Enter a valid price (0 or more).

Total: ₹0.00

TC-08 – Both quantity and price invalid at the same time

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Wireless Mouse

Quantity

-2

Enter a valid quantity (0 or more).

Price per unit (₹)

e.g. 50

Enter a valid price (0 or more).

Total: ₹0.00

TC-09 – Boundary case: quantity 0 is accepted, total ₹0.00, no error

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Wireless Mouse

Quantity

0

Price per unit (₹)

750

Wireless Mouse Total: ₹0.00

TC-10 – Decimal price rounded to two decimal places

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

USB Cable

Quantity

3

Price per unit (₹)

99.99

USB Cable Total: ₹299.97

TC-11 – Blank product name falls back to "Product"

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Quantity

Price per unit (₹)

Product Total: ₹360.00

TC-12 – Price re-entered after being cleared; error clears and total recalculates

Interactive Product Calculator

file:///C:/FSWD/task-05/01-product-calculator/index.html

Product Calculator

Product Name

Quantity

Price per unit (₹)

USB Cable Total: ₹360.00

10. RESULT

Thus, an interactive product calculator was successfully built using JavaScript functions, event listeners and DOM updates. The total price was calculated and displayed dynamically whenever the product name, quantity or price changed, and the page was never reloaded. Invalid quantity and price values were each validated independently and reported with their own red error message, and the total was reset to ₹0.00 whenever either field was invalid. A blank or spaces-only product name was handled gracefully by falling back to the label "Product" rather than leaving the result line without a name. One behaviour worth noting is that the quantity check uses `quantity < 0` rather than `quantity <= 0`,

so a quantity of exactly zero is accepted as valid and produces a total of ₹0.00 without an error message; tightening this to ≤ 0 would treat a zero quantity the same way as a negative one.

EXPERIMENT 2

1. TITLE

Node.js File Management Application Using the File System (fs) Module and Command-Line Arguments.

2. AIM

To create a Node.js application that accepts a filename and initial content as command-line arguments and performs create/write, read, append and final display operations on that file using the asynchronous File System module.

3. OBJECTIVE

- To take the filename and the initial content from `process.argv`, the array of command-line arguments passed to `node app.js`, instead of prompting interactively.
- To fall back to sensible default values, `sample.txt` and a default sentence, when the filename or content is not supplied on the command line.
- To build a full file path with `path.join(__dirname, fileName)` so the file is always created next to the script regardless of the current working directory.
- To create and write a file using `fs.writeFile()`.
- To read the contents of a file using `fs.readFile()` with utf8 encoding.
- To add further content to an existing file using `fs.appendFile()`, with a leading newline so the appended text starts on its own line.
- To display the final contents of the file after the append operation.
- To handle errors such as an invalid folder path using the error object supplied to each callback, printing `err.message` and stopping without crashing the process.

4. THEORY

Node.js allows JavaScript to run outside the browser, on the server side. The File System module, imported with `require("fs")`, gives JavaScript direct access to files on the machine, which is not possible in a browser. This version of the program takes its input from `process.argv` rather than from the `readline` module: `process.argv[0]` is the path to the Node.js executable, `process.argv[1]` is the path to the script itself, and every argument after that is available as `process.argv[2]`, `process.argv[3]` and so on, which is why the filename is read as `process.argv[2]` and the initial content as `process.argv[3]`.

The asynchronous versions of the fs methods are used throughout. An asynchronous method does not stop the program while the disk operation takes place; instead, it accepts a callback function that Node.js runs once the operation has finished, with the error object as its first argument. Because each step depends on the result of the previous one, the four operations are organised as four named functions, `createFile()`, `readFile()`, `appendFile()` and `displayFinalContent()`, where each function calls the next one from inside its own success callback. This keeps the nesting shallow and gives each stage of the pipeline a clear, self-describing name in the console output.

The key methods and concepts used in this program are:

- `require("fs")` imports the built-in File System module, and `require("path")` imports the module used to build file paths safely across operating systems.

- `process.argv[2]` and `process.argv[3]` read the filename and the initial content typed after `node app.js` on the command line; `|| 'default'` supplies a fallback when they are omitted.
- `path.join(__dirname, fileName)` builds an absolute path to the file inside the same folder as `app.js`, regardless of where the command is run from.
- `fs.writeFile(path, content, callback)` creates the file, or replaces its contents if the file already exists.
- `fs.readFile(path, "utf8", callback)` reads the file and returns its contents as a string; without "utf8" a Buffer would be returned instead.
- `fs.appendFile(path, text, callback)` adds text to the end of the file without deleting what is already there; a leading `\n` makes the newly appended text start on a new line.
- `err.message` prints a readable description of a failure, such as `ENOENT` when the target folder does not exist.
- Each stage function calls the next stage only from inside the success branch of its own callback, which is what keeps the four disk operations running strictly in order: write, then read, then append, then read again.

The basic flow of the program is:

The script starts with `node app.js <filename> <content>` → `createFile()` writes the file with the supplied (or default) content → on success, `readFile()` reads it back and prints the current contents → `appendToFile()` appends a fixed follow-up line with a leading newline → on success, `displayFinalContent()` reads the file one last time and prints the final contents → if any step fails, its error message is printed and the chain stops there without running the remaining steps.

5. ALGORITHM / PROCEDURE

- Create a project folder and a single file, `app.js`, that will hold the whole pipeline.
- Import `fs` and `path` with `require()`.
- Read the filename from `process.argv[2]`, defaulting to `sample.txt` when it is not supplied, and the initial content from `process.argv[3]`, defaulting to a fixed sentence.
- Build the full file path with `path.join(__dirname, fileName)`.
- Define `createFile()`, which calls `fs.writeFile()` with the initial content; on success it logs a confirmation and calls `readFile()`.
- Define `readFile()`, which calls `fs.readFile()` with `utf8` encoding, prints the current contents, and calls `appendToFile()` on success.
- Define `appendToFile()`, which calls `fs.appendFile()` with a newline followed by a fixed extra sentence, and calls `displayFinalContent()` on success.
- Define `displayFinalContent()`, which reads the file one more time and prints it as the final contents, ending the pipeline.
- Every function checks its callback's error argument first, printing `err.message` and returning immediately if it is set, so a failure at any stage stops the chain instead of continuing with a missing file.
- Call `createFile()` at the bottom of the script to start the pipeline.
- Run the program with `node app.js`, and also test it by passing an explicit filename and content, an existing filename (to see the overwrite behaviour), and a filename inside a folder that does not exist.

6. PROGRAM

app.js

```
const fs = require('fs');
const path = require('path');

// Accept filename & initial content from command-line args, or use defaults
const fileName = process.argv[2] || 'sample.txt';
const initialContent = process.argv[3] || 'Hello, this is the initial content of the file.';
const appendContent = '\nThis line was appended later using fs.appendFile().';

const filePath = path.join(__dirname, fileName);

/**
 * Step 1: Create / write the file
 */
function createFile() {
  console.log(`\nStep 1: Writing to "${fileName}"...`);
  fs.writeFile(filePath, initialContent, (err) => {
    if (err) {
      console.error('Error writing file:', err.message);
      return;
    }
    console.log('File created/written successfully.');
```

```
    readFile(); // proceed to next step
  });
}

/**
 * Step 2: Read the file's contents
 */
function readFile() {
  console.log(`\nStep 2: Reading "${fileName}"...`);
  fs.readFile(filePath, 'utf8', (err, data) => {
    if (err) {
      console.error('Error reading file:', err.message);
      return;
    }
    console.log('Current contents:\n---');
```

```
    console.log(data);
    console.log('---');
    appendToFile(); // proceed to next step
  });
}

/**
 * Step 3: Append additional content
 */
function appendToFile() {
  console.log(`\nStep 3: Appending content to "${fileName}"...`);
  fs.appendFile(filePath, appendContent, (err) => {
    if (err) {
      console.error('Error appending to file:', err.message);
      return;
    }
    console.log('Content appended successfully.');
```

```
    displayFinalContent(); // proceed to final step
  });
}

/**
 * Step 4: Display the final contents after append
 */
function displayFinalContent() {
  console.log(`\nStep 4: Final contents of "${fileName}"...`);
  fs.readFile(filePath, 'utf8', (err, data) => {
    if (err) {
      console.error('Error reading file:', err.message);
      return;
    }
    console.log('Final contents:\n---');
```

```
    console.log(data);
    console.log('---\nDone.');
```

```
  });
}
createFile();
```

7. CODE EXPLANATION

Code	Explanation
<code>require("fs") / require("path")</code>	Imports the File System module and the path-building module used to construct a safe absolute file path.
<code>process.argv[2] / process.argv[3]</code>	Reads the filename and the initial content passed on the command line after <code>node app.js</code> ; the <code> </code> fallback supplies defaults when they are omitted.
<code>path.join(__dirname, fileName)</code>	Builds the full path to the target file inside the folder that contains <code>app.js</code> .
<code>fs.writeFile()</code>	Creates the file with the given content, or overwrites it if the file already exists.
<code>fs.readFile(path, "utf8", cb)</code>	Reads the file and returns its contents as a readable string instead of a Buffer.
<code>fs.appendFile()</code>	Adds the fixed follow-up sentence to the end of the file on a new line, keeping the existing content.
<code>createFile() → readFile() → appendToFile() → displayFinalContent()</code>	Each function starts the next one from inside its own success callback, which is what keeps the four operations running in the correct order.
<code>if (err) { console.error(...); return; }</code>	The error-first check repeated in every callback; it stops the pipeline and reports the failure instead of continuing with a missing or unwritten file.
<code>err.message</code>	Prints a readable description of the failure, such as a missing folder.

8. TEST CASES AND EXECUTION DATA

The program was run from the terminal with `node app.js`, with the filename and content supplied as command-line arguments. The screenshots below are the actual terminal output captured from each run, including the files it produced on disk.

Test Case	Action / Input	Expected Result	Actual Result
TC-01	Run <code>node app.js</code> with no arguments	<code>sample.txt</code> is created with the default sentence, read, appended to, and the final contents show both lines	All four steps completed on <code>sample.txt</code> — Pass
TC-02	<code>node app.js notes.txt "This is the first line of the file."</code>	<code>notes.txt</code> is created with that content, then the fixed line is appended	All four steps completed on <code>notes.txt</code> — Pass
TC-03	<code>node app.js notes.txt "Overwritten content." (re-run on the existing file)</code>	The old content of <code>notes.txt</code> is replaced, not merged, before the append happens	File overwritten, then appended — Pass
TC-04	<code>node app.js nofolder/data.txt "Some content" (folder does not exist)</code>	<code>writeFile</code> fails with an <code>ENOENT</code> error message and the pipeline stops after step 1	<code>ENOENT</code> error printed, no further steps ran — Pass
TC-05	<code>node app.js "" "Some content" (empty filename argument)</code>	The empty string falls back to the default filename <code>sample.txt</code>	Wrote to <code>sample.txt</code> using the fallback name — Pass
TC-06	<code>node app.js empty.txt (filename given, content omitted)</code>	The missing content argument falls back to the default sentence	<code>empty.txt</code> created with the default sentence — Pass

Test Case	Action / Input	Expected Result	Actual Result
TC-07	node app.js words.txt Hello world (content given as two separate arguments)	Only the first extra argument, "Hello", is used as the content; "world" is ignored	words.txt created with just "Hello" as content — Pass

9. OUTPUT

TC-01 – node app.js run with no arguments: default filename and content

```

Terminal
02-node-file-management$ node app.js
Step 1: Writing to "sample.txt"...
File created/written successfully.

Step 2: Reading "sample.txt"...
Current contents:
---
Hello, this is the initial content of the file.
---

Step 3: Appending content to "sample.txt"...
Content appended successfully.

Step 4: Final contents of "sample.txt"...
Final contents:
---
Hello, this is the initial content of the file.
This line was appended later using fs.appendFile().
---
Done.

```

TC-02 – Filename and content supplied on the command line

```

Terminal
02-node-file-management$ node app.js notes.txt "This is the first line of the file."
Step 1: Writing to "notes.txt"...
File created/written successfully.

Step 2: Reading "notes.txt"...
Current contents:
---
This is the first line of the file.
---

Step 3: Appending content to "notes.txt"...
Content appended successfully.

Step 4: Final contents of "notes.txt"...
Final contents:
---
This is the first line of the file.
This line was appended later using fs.appendFile().
---
Done.

```

TC-03 – Re-run on the existing notes.txt: old content is overwritten, then appended

```
Terminal

02-node-file-management$ node app.js notes.txt "Overwritten content."
Step 1: Writing to "notes.txt"...
File created/written successfully.

Step 2: Reading "notes.txt"...
Current contents:
---
Overwritten content.
---

Step 3: Appending content to "notes.txt"...
Content appended successfully.

Step 4: Final contents of "notes.txt"...
Final contents:
---
Overwritten content.
This line was appended later using fs.appendFile().
---
Done.
```

TC-04 – Invalid input: target folder does not exist, ENOENT error

```
Terminal

02-node-file-management$ node app.js nofolder/data.txt "Some content"
Step 1: Writing to "nofolder/data.txt"...
Error writing file: ENOENT: no such file or directory, open '/tmp/fswd/task-05/02-node-file-management/nofolder/data.txt'
```

TC-05 – Empty filename argument falls back to sample.txt

```
Terminal

02-node-file-management$ node app.js "" "Some content"
Step 1: Writing to "sample.txt"...
File created/written successfully.

Step 2: Reading "sample.txt"...
Current contents:
---
Some content
---

Step 3: Appending content to "sample.txt"...
Content appended successfully.

Step 4: Final contents of "sample.txt"...
Final contents:
---
Some content
This line was appended later using fs.appendFile().
---
Done.
```

TC-06 – Filename given, content omitted: falls back to the default sentence

```
Terminal

02-node-file-management$ node app.js empty.txt
Step 1: Writing to "empty.txt"...
File created/written successfully.

Step 2: Reading "empty.txt"...
Current contents:
---
Hello, this is the initial content of the file.
---

Step 3: Appending content to "empty.txt"...
Content appended successfully.

Step 4: Final contents of "empty.txt"...
Final contents:
---
Hello, this is the initial content of the file.
This line was appended later using fs.appendFile().
---
Done.
```

TC-07 – Extra command-line word beyond the content argument is ignored

```
Terminal

02-node-file-management$ node app.js words.txt Hello world
Step 1: Writing to "words.txt"...
File created/written successfully.

Step 2: Reading "words.txt"...
Current contents:
---
Hello
---

Step 3: Appending content to "words.txt"...
Content appended successfully.

Step 4: Final contents of "words.txt"...
Final contents:
---
Hello
This line was appended later using fs.appendFile().
---
Done.
```

10. RESULT

Thus, a Node.js file management application was successfully created using the File System module, driven entirely by command-line arguments rather than interactive prompts. The program created and wrote the file, read and displayed its contents, appended additional content and displayed the final contents, chaining the four asynchronous steps through named functions. Invalid input was handled correctly: a path pointing at a folder that does not exist produced an ENOENT error from `writeFile` and stopped the pipeline instead of crashing the process, while a missing filename or content argument fell back to the built-in defaults rather than failing. One point noted during testing is that `fs.writeFile()` silently replaces an existing file's content, so re-running the program with the same

filename loses whatever was written there before; prompting for confirmation, or switching to an append-only mode for repeat runs, would make the tool safer to use on files that already matter.

EXPERIMENT 3

1. TITLE

Express.js Student Server with Routing, Route Parameters and JSON Responses.

2. AIM

To create a basic Express.js server exposing the routes `/`, `/students`, `/students/:id` and `/about`, where `/students` returns a JSON list of five students, `/students/:id` looks a single student up by id, and `/about` returns a JSON description of the application.

3. OBJECTIVE

- To install Express.js in a Node.js project using npm and import it with `require("express")`.
- To create an Express application object and start a server on a chosen port with `app.listen()`.
- To register the built-in `express.json()` middleware with `app.use()` so the server is ready to parse a JSON request body if one is sent.
- To define GET routes for `/`, `/students` and `/about` using `app.get()`, and a parameterised route, `/students/:id`, that reads the id from `req.params`.
- To send an HTML response using `res.send()` and a JSON response using `res.json()`.
- To set an HTTP status code explicitly using `res.status()`, including a 404 for a student id that is not found.
- To handle requests to routes that do not exist, and requests using a method that is not defined for an existing path, by falling through to a catch-all 404 handler.
- To verify every route, including the not-found cases, using a browser and using curl.

4. THEORY

Express.js is a web application framework built on top of the Node.js http module. It removes the repetitive work of parsing URLs and methods by hand and lets the developer describe the server as a set of routes. A route is a combination of an HTTP method and a path; when a request arrives, Express matches it against the routes in the order they were defined and runs the first handler that matches.

Every route handler receives a request object, `req`, and a response object, `res`. A response must be sent exactly once per request, which is why each handler here ends with a single `res.send()` or `res.json()` call. `res.json()` converts a JavaScript value into JSON text and sets the Content-Type header to `application/json` automatically, which is why it is used for `/students`, `/students/:id` and `/about`, while the home route uses `res.send()` to return a short HTML snippet instead.

The route `/students/:id` introduces a route parameter: the colon before `id` tells Express to treat that part of the path as a variable and make it available as `req.params.id`. Because that value always arrives as a string, it is converted with `parseInt()` before being compared against the numeric `id` field on each student object. When no student matches, the handler returns a 404 response with a JSON error body instead of falling through to the catch-all handler, which is why `/students/:id` can produce its own not-found message rather than the generic one.

The `app.use()` call placed after every other route acts as a catch-all: Express only reaches it when nothing earlier has matched the request, so it is the natural place to send a generic 404 response. Its position in the file matters, since a request is matched against the routes in the order they were

registered, and a route with a matching path but a different HTTP method, such as a POST to /students, is treated exactly like a path that does not exist at all — it falls through to the same catch-all.

The key methods and concepts used in this program are:

- `require("express")` imports the Express framework, which must first be installed with `npm install express`.
- `const app = express()` creates the application object that holds all the routes.
- `app.use(express.json())` registers middleware that parses an incoming JSON request body before any route handler runs.
- `app.get(path, handler)` registers a handler for GET requests to that path.
- `req.params.id` reads the dynamic segment of a parameterised route such as `/students/:id`.
- `res.send(text)` sends a plain text or HTML response to the client.
- `res.json(value)` sends a JavaScript object or array as JSON, setting the correct content type automatically.
- `res.status(code)` sets the HTTP status code before the response body is sent, used here for the 200 success responses and for the 404 responses.
- `app.use(handler)` placed after all the routes runs only when nothing else matched, and is used here as the generic 404 handler.
- `app.listen(PORT, callback)` starts the server and runs the callback once it is listening.

The basic flow of the program is:

The server is started with `node index.js` → Express listens on port 3000 → a browser, curl or Postman sends a request → Express compares the method and the path with the registered routes in order → / returns the welcome HTML, /students returns the JSON array, /students/:id returns one matching student or a 404 JSON error, /about returns the JSON description → any other path, or a request using a method with no matching route, falls through to the catch-all and returns "404 - Route Not Found".

5. ALGORITHM / PROCEDURE

- Create a project folder and run `npm init -y` to generate `package.json`, then install Express with `npm install express`.
- Create `index.js` and import Express with `require("express")`; create the application object and store the port number 3000 in a constant.
- Call `app.use(express.json())` so the server can parse a JSON body on any future route that needs one.
- Define an array named `students` containing five objects, each with `id`, `name`, `course` and `year`.
- Define a GET route for `/` that sends a short welcome HTML snippet using `res.send()`.
- Define a GET route for `/students` that sends `{ success, count, data }` using `res.json()`, where `data` is the `students` array.
- Define a GET route for `/students/:id` that looks the `id` up in the array with `.find()`; if nothing matches, respond with `res.status(404).json({ success: false, message: ... })`, otherwise respond with `res.status(200).json({ success: true, data: student })`.
- Define a GET route for `/about` that sends a JSON object describing the application, using `res.status(200).json()`.
- After all the routes, add `app.use()` that sets the status to 404 and sends "404 - Route Not Found".

- Call `app.listen(PORT, callback)` and print the server address and the list of available routes from the callback.
- Start the server with `node index.js`, open each route in a browser, and use curl or Postman to check `/students/99` (a missing id), `/teachers` (an undefined path) and a POST to `/students` (an undefined method for an existing path).

6. PROGRAM

index.js

```
const express = require('express');
const app = express();
const PORT = 3000;

app.use(express.json());

// Sample in-memory student data
const students = [
  { id: 1, name: 'Aarav Sharma', course: 'B.Tech CSE', year: 2 },
  { id: 2, name: 'Priya Reddy', course: 'B.Tech IT', year: 3 },
  { id: 3, name: 'Rahul Verma', course: 'BCA', year: 1 },
  { id: 4, name: 'Sneha Iyer', course: 'B.Tech ECE', year: 4 },
  { id: 5, name: 'Karthik Nair', course: 'B.Sc CS', year: 2 }
];

// Home route
app.get('/', (req, res) => {
  res.send('<h1>Welcome to the Student Server</h1><p>Try /students or /about</p>');
});

// GET /students - returns list of students as JSON
app.get('/students', (req, res) => {
  res.status(200).json({
    success: true,
    count: students.length,
    data: students
  });
});

// GET /students/:id - bonus route showing route parameters + HTTP status handling
app.get('/students/:id', (req, res) => {
  const student = students.find(s => s.id === parseInt(req.params.id));
  if (!student) {
    return res.status(404).json({ success: false, message: 'Student not found' });
  }
  res.status(200).json({ success: true, data: student });
});

// GET /about - information about the application
app.get('/about', (req, res) => {
  res.status(200).json({
    appName: 'Student Server API',
    description: 'A basic Express.js application demonstrating routing, HTTP methods, and response handling.',
    version: '1.0.0',
    author: 'Govi'
  });
});

// Catch-all 404 handler for undefined routes
app.use((req, res) => {
  res.status(404).send('404 - Route Not Found');
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
  console.log('Available routes: /, /students, /students/:id, /about');
});
```

7. CODE EXPLANATION

Code	Explanation
<code>require("express")</code>	Imports the Express framework installed through npm.
<code>const app = express()</code>	Creates the Express application object on which the routes are registered.
<code>app.use(express.json())</code>	Registers middleware that parses a JSON request body before any route handler sees the request.
<code>const students = [...]</code>	An array of five student objects, each with an id, a name, a course and a year.
<code>app.get("/", handler)</code>	Handles GET requests to the home route and sends a short welcome HTML snippet.
<code>app.get("/students", handler)</code>	Handles GET requests to /students and returns the full student list wrapped in a success/count/data object.
<code>app.get("/students/:id", handler)</code>	Handles GET requests to a specific student id; looks it up with <code>.find()</code> and returns 404 JSON when no student matches.
<code>req.params.id/parseInt()</code>	Reads the id segment from the URL as a string and converts it to a number before comparing it with each student's id.
<code>res.json(value)</code>	Converts the given object or array into JSON and sets the Content-Type header to application/json.
<code>app.get("/about", handler)</code>	Handles GET requests to /about and sends a JSON description of the application.
<code>app.use((req, res) => ...)</code>	Catch-all handler placed after all routes; it runs only when no route matched the method and path.
<code>res.status(404)</code>	Sets the HTTP status code to 404 before sending the not-found message.
<code>app.listen(PORT, callback)</code>	Starts the server and prints the address and the available routes once it is listening.

8. TEST CASES AND EXECUTION DATA

The server was started from the terminal with `node index.js` and every route was verified with a real browser (Playwright/Chromium) and with curl, including a route that does not exist, a route parameter that does not match any student, and an HTTP method that is not defined for an existing path.

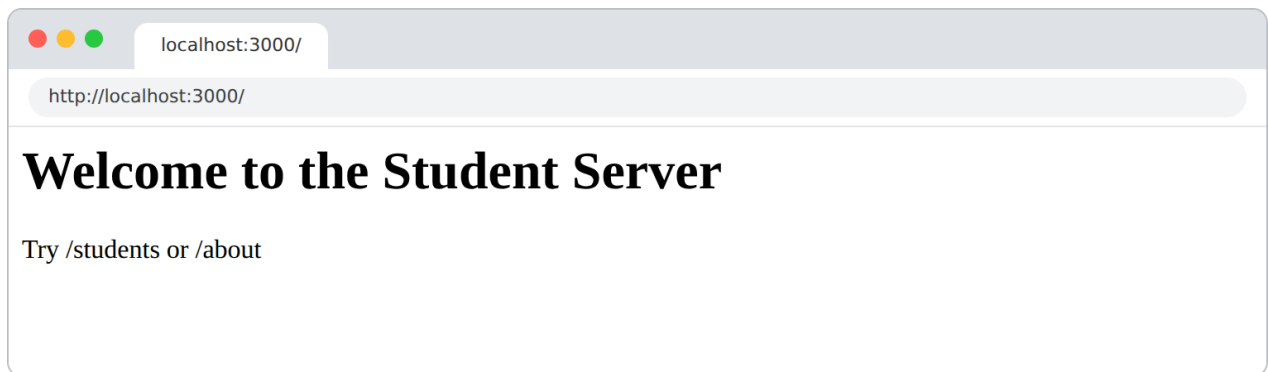
Test Case	Action / Input	Expected Result	Actual Result
TC-01	Run <code>node index.js</code> in the terminal	The startup message and the list of available routes are printed	Server running at http://localhost:3000 — Pass
TC-02	Open <code>http://localhost:3000/</code> in a browser	The welcome HTML "Welcome to the Student Server" is displayed	Welcome message displayed, status 200 — Pass
TC-03	Open <code>http://localhost:3000/students</code>	A JSON object with success, count 5 and a data array of five students is returned	JSON list of five students, status 200 — Pass
TC-04	Open <code>http://localhost:3000/students/3</code>	The single student with id 3 (Rahul Verma) is returned as JSON	<code>{"success":true,"data":{"id":3,"name":"Rahul Verma",...}}</code> , status 200 — Pass
TC-05	GET <code>http://localhost:3000/students/99</code> (id does not exist)	A 404 JSON response <code>{"success":false,"message":"Student not found"}</code> is returned	404 JSON not-found response — Pass
TC-06	Open <code>http://localhost:3000/about</code>	A JSON object describing the application (name, description, version, author) is returned	About JSON displayed, status 200 — Pass
TC-07	Open <code>http://localhost:3000/teachers</code> (undefined route)	The catch-all handler returns "404 - Route Not Found"	404 - Route Not Found, status 404 — Pass
TC-08	<code>curl -X POST http://localhost:3000/students</code> (method not defined for this path)	The request matches no route and falls through to the catch-all 404 handler	404 - Route Not Found, status 404 — Pass

9. OUTPUT

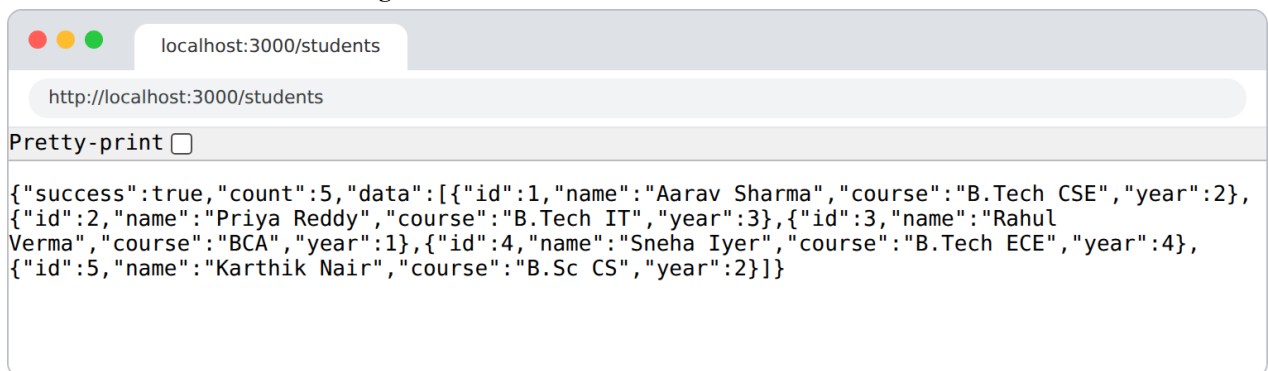
TC-01 – Server started from the terminal, listing the available routes

```
Terminal
03-student-server$ node index.js
Server running at http://localhost:3000
Available routes: /, /students, /students/:id, /about
```

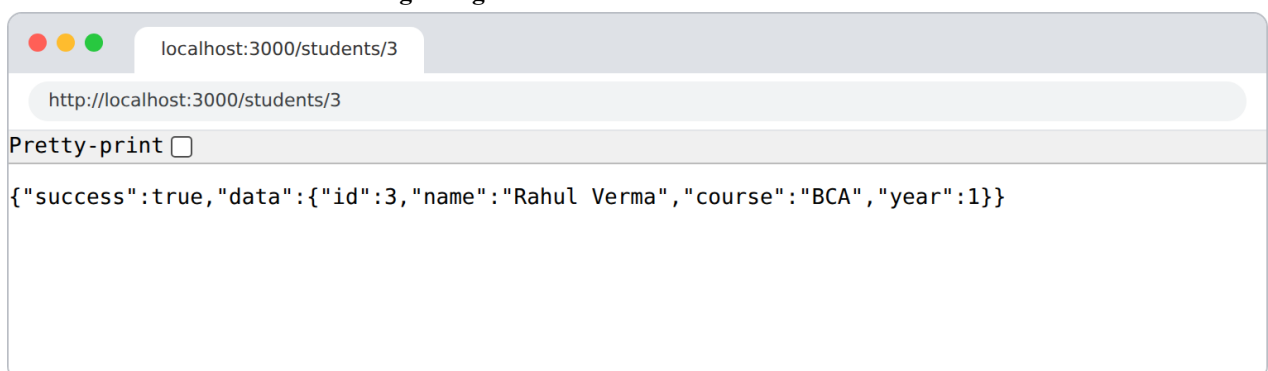
TC-02 – Home route / in the browser



TC-03 – /students route returning the JSON student list



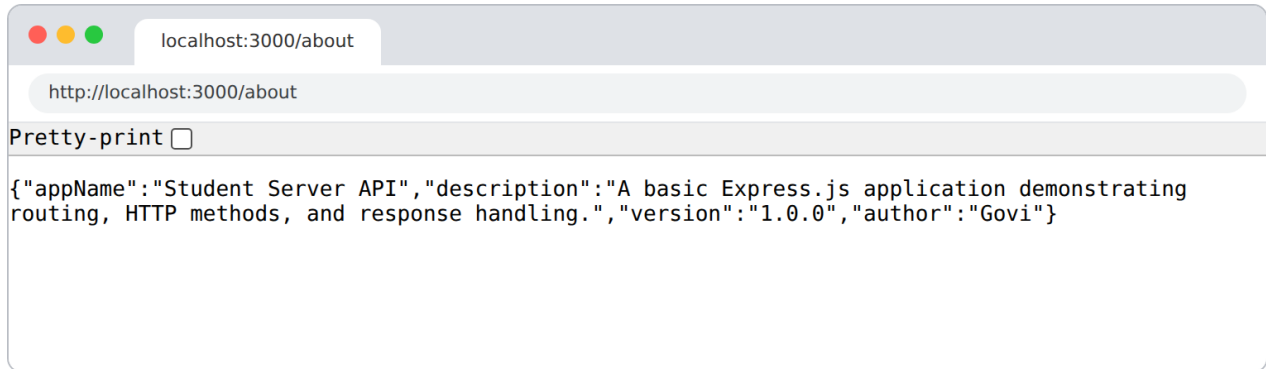
TC-04 – /students/3 route returning a single matched student



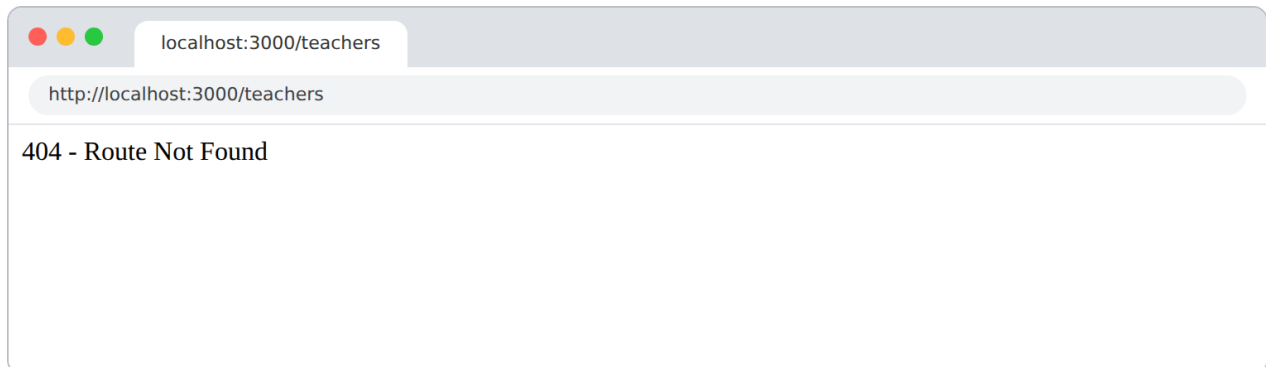
TC-05 – /students/99: id not found, 404 JSON error response

```
Terminal
03-student-server$ curl -s -w "\nHTTP %{http_code}\n" http://localhost:3000/students/99
{"success":false,"message":"Student not found"}
HTTP 404
```

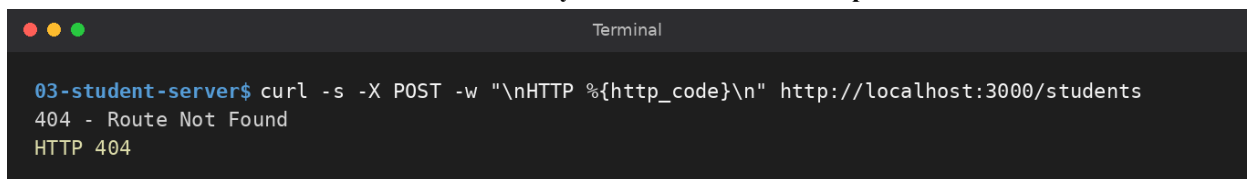
TC-06 – /about route returning the application description as JSON



TC-07 – Invalid route /teachers handled by the catch-all 404 handler



TC-08 – POST /students returns 404 because only GET is defined for that path



10. RESULT

Thus, a basic Express.js student server was successfully created with the routes `/`, `/students`, `/students/:id` and `/about`. The home route returned an HTML snippet using `res.send()`, while `/students`, `/students/:id` and `/about` all returned structured JSON using `res.json()`, each verified in the browser and with curl. A request for a student id that does not exist was handled inside the `/students/:id` route itself with a dedicated 404 JSON error, while requests to an undefined path and requests using an HTTP method with no matching route (a POST to `/students`) were both handled by the shared catch-all middleware and returned "404 - Route Not Found" with the correct status code. A possible improvement is to add POST, PUT and DELETE routes so that students can also be added, updated and removed rather than only listed and looked up.

EXPERIMENT 4

1. TITLE

Demonstration of the Document Object Model (DOM): Changing the Content, Style and Attributes of an HTML Element Using Separate Buttons.

2. AIM

To explain the Document Object Model and to demonstrate how JavaScript can dynamically change the content, the style and the attributes of an HTML element at runtime, using three independent buttons plus a fourth button that resets the element back to its original state.

3. OBJECTIVE

- To understand the structure of the Document Object Model and how the browser represents an HTML page as a tree of nodes.
- To access a target element on the page from JavaScript using `document.getElementById()`.
- To change the visible text of the element using the `textContent` property, from a "Change Content" button.
- To change the appearance of the element at runtime using the style object, from a separate "Change Style" button.
- To add or modify attributes of the element using `setAttribute()`, from a separate "Change Attribute" button, including a custom `data-modified` attribute.
- To provide a "Reset" button that restores the element's text, removes the inline style and the custom attribute, and puts the title attribute back to its original value.
- To verify the changes visually in the browser, through the tooltip, and in the browser developer tools, and to confirm that refreshing the page restores the original DOM.

4. THEORY

When a browser loads an HTML page, it does not keep the file as plain text. It parses the file and builds an object-based representation of it in memory called the Document Object Model, or DOM. In this model every element, every attribute and every piece of text becomes a node, and the nodes are arranged as a tree with the document object at the root, `<html>` below it, then `<head>` and `<body>`, and so on down to the individual elements.

The value of the DOM is that it is a live model. It is not a copy of the HTML file; it is what the browser actually displays. Whenever JavaScript changes a node in the DOM, the browser immediately repaints the page to match, which is how a page can change without being reloaded and without the original HTML file being edited in any way. Reloading the page rebuilds the DOM from the file, so all such changes are lost, which is exactly what the "refresh the page" test case in this experiment demonstrates.

This version of the demonstration splits the three kinds of change across three separate buttons, each wired to its own `onclick` handler function, rather than combining all three changes into one click handler:

- Content – `changeContent()` replaces the text inside the target paragraph using `textContent`, which treats the value as plain text and does not interpret HTML tags, making it the safer choice over `innerHTML`.

- **Style** – `changeStyle()` sets four properties on the style object at once: `backgroundColor`, `color`, `fontWeight` and `border`; CSS properties written with a hyphen, such as `background-color`, become camelCase in JavaScript, so it is written as `backgroundColor`.
- **Attributes** – `changeAttribute()` calls `setAttribute("title", ...)` to change the tooltip text, re-sets the `id` attribute to the same value to show that `setAttribute` can be called on any attribute including `id`, and adds a new custom attribute, `data-modified="true"`, that did not exist in the original markup at all.

A fourth function, `resetAll()`, is the counterpart to the other three: it puts the text back to its original sentence, calls `removeAttribute("style")` to drop every inline style property in one call rather than resetting each one individually, restores the original title, and calls `removeAttribute("data-modified")` to remove the attribute that `changeAttribute()` had added. Because all four functions operate on the same constant, target, obtained once with `document.getElementById()` at the top of the script, none of the buttons needs to look the element up again.

The key methods and concepts used in this program are:

- `document.getElementById("target")` returns the node in the DOM tree whose `id` attribute matches, and is looked up once and reused by every handler.
- An inline `onclick="functionName()"` attribute on each button registers that function to run when the button is clicked, instead of when the page loads.
- `element.textContent` replaces the text inside an element with plain text, safely, without interpreting any HTML it might contain.
- `element.style.backgroundColor`, `.color`, `.fontWeight` and `.border` set inline CSS on the element, which overrides any styles defined in the stylesheet.
- `element.setAttribute(name, value)` adds or updates an attribute on the element, including custom `data-*` attributes and the standard `title` attribute, which the browser shows as a tooltip on hover.
- `element.removeAttribute(name)` deletes an attribute from the element entirely; used on `"style"` to clear every inline style property in one call and on `"data-modified"` to remove the custom attribute added earlier.
- The changes exist only in the DOM in memory, so refreshing the page restores the original HTML.

The basic flow of the page is:

The page loads and the browser builds the DOM → the target paragraph is selected once and stored in a constant → the user clicks one of the four buttons → the matching handler runs → Change Content replaces the text, Change Style sets four inline style properties, Change Attribute updates the title, re-sets the `id` and adds a new `data-modified` attribute, and Reset undoes all of the above → the browser repaints the page immediately after each click, without reloading it, until the page is refreshed and the DOM is rebuilt from the original file.

5. ALGORITHM / PROCEDURE

- Create a single HTML file containing a card with a heading, a `<p>` element with `id="target"` and an initial `title="original tooltip"`, and four `<button>` elements.
- Give each button an inline `onclick` attribute pointing at its own function: `changeContent()`, `changeStyle()`, `changeAttribute()` and `resetAll()`.
- Write the JavaScript inside a `<script>` tag placed at the end of the body, so the target element already exists in the DOM when the script runs.

- Select the target paragraph once using `document.getElementById()` and store it in a constant.
- Define `changeContent()`, which sets `target.textContent` to a new sentence.
- Define `changeStyle()`, which sets `target.style.backgroundColor`, `.color`, `.fontWeight` and `.border` to new values.
- Define `changeAttribute()`, which calls `target.setAttribute()` for "title" and "id", and adds the new "data-modified" attribute set to "true".
- Define `resetAll()`, which restores the original text, calls `target.removeAttribute("style")`, restores the original title with `setAttribute()`, and calls `target.removeAttribute("data-modified")`.
- Save the file and open it in a browser; note the original content, then click Change Content, Change Style and Change Attribute in turn (each independently, and then together) and observe the effect of each one.
- Hover the mouse over the paragraph after Change Attribute to confirm that the tooltip text has changed.
- Open the browser developer tools and inspect the target element to confirm the inline style and the new attributes on the node in the DOM.
- Click Reset and confirm the element returns to exactly its original outerHTML; separately, refresh the page after making changes and confirm the DOM is rebuilt from the file with the original content restored.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>DOM Demo - Dynamic Content, Style, and Attributes</title>
<style>
  body { font-family: Arial, sans-serif; background: #f4f6f8; padding: 40px; }
  .card { max-width: 500px; margin: 0 auto; background: #fff; padding: 25px 30px;
    border-radius: 10px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); }
  #target {
    padding: 15px; border: 2px solid #ccc; border-radius: 6px; margin: 15px 0;
    transition: all 0.3s ease;
  }
  button {
    padding: 8px 14px; margin: 4px 4px 4px 0; border: none; border-radius: 5px;
    background: #2c7be5; color: white; cursor: pointer; font-size: 14px;
  }
  button:hover { background: #1a5bb8; }
</style>
</head>
<body>

<div class="card">
  <h2>DOM Manipulation Demo</h2>

  <!-- This <p> element is what we will manipulate -->
  <p id="target" title="original tooltip">I am a paragraph element. Click the buttons below to
change my content, style, and attributes.</p>

  <button onclick="changeContent()">Change Content</button>
  <button onclick="changeStyle()">Change Style</button>
  <button onclick="changeAttribute()">Change Attribute</button>
  <button onclick="resetAll()">Reset</button>
</div>

<script>
  const target = document.getElementById('target');

  // 1. Change CONTENT
  function changeContent() {
    target.textContent = "My content has been changed dynamically using JavaScript!";
```

```

    }

    // 2. Change STYLE
    function changeStyle() {
        target.style.backgroundColor = "#dff0d8";
        target.style.color = "#3c763d";
        target.style.fontWeight = "bold";
        target.style.border = "2px solid #3c763d";
    }

    // 3. Change ATTRIBUTE
    function changeAttribute() {
        target.setAttribute('title', 'updated tooltip - try hovering!');
        target.setAttribute('id', 'target'); // id stays same, but shows attribute access
        target.setAttribute('data-modified', 'true');
    }

    // Reset everything back to original
    function resetAll() {
        target.textContent = "I am a paragraph element. Click the buttons below to change my
content, style, and attributes.";
        target.removeAttribute('style');
        target.setAttribute('title', 'original tooltip');
        target.removeAttribute('data-modified');
    }
}
</script>

</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById("target")</code>	Selects the target <code><p></code> node from the DOM once, so every handler function can reuse the same reference.
<code>onclick="changeContent()"</code> etc.	Wires each button directly to its own handler function, rather than one shared click listener for every change.
<code>changeContent() → target.textContent = "..."</code>	Changes the content: replaces the visible text inside the paragraph.
<code>changeStyle() → target.style.backgroundColor / .color / .fontWeight / .border</code>	Changes the style: applies four inline CSS properties to the paragraph at once.
<code>changeAttribute() → target.setAttribute("title", ...)</code>	Changes the attributes: updates the title attribute, which the browser displays as a tooltip on hover.
<code>target.setAttribute("id", "target")</code>	Re-sets the id attribute to demonstrate that <code>setAttribute</code> works on any attribute, not only custom ones.
<code>target.setAttribute("data-modified", "true")</code>	Adds a brand-new custom attribute that was not present in the original markup at all.
<code>resetAll() → target.textContent = "..."</code>	Restores the original sentence, undoing <code>changeContent()</code> .
<code>target.removeAttribute("style")</code>	Removes every inline style property in a single call, undoing <code>changeStyle()</code> .
<code>target.setAttribute("title", "original tooltip")</code>	Restores the original tooltip text, undoing part of <code>changeAttribute()</code> .
<code>target.removeAttribute("data-modified")</code>	Removes the custom attribute that <code>changeAttribute()</code> had added, completing the undo.

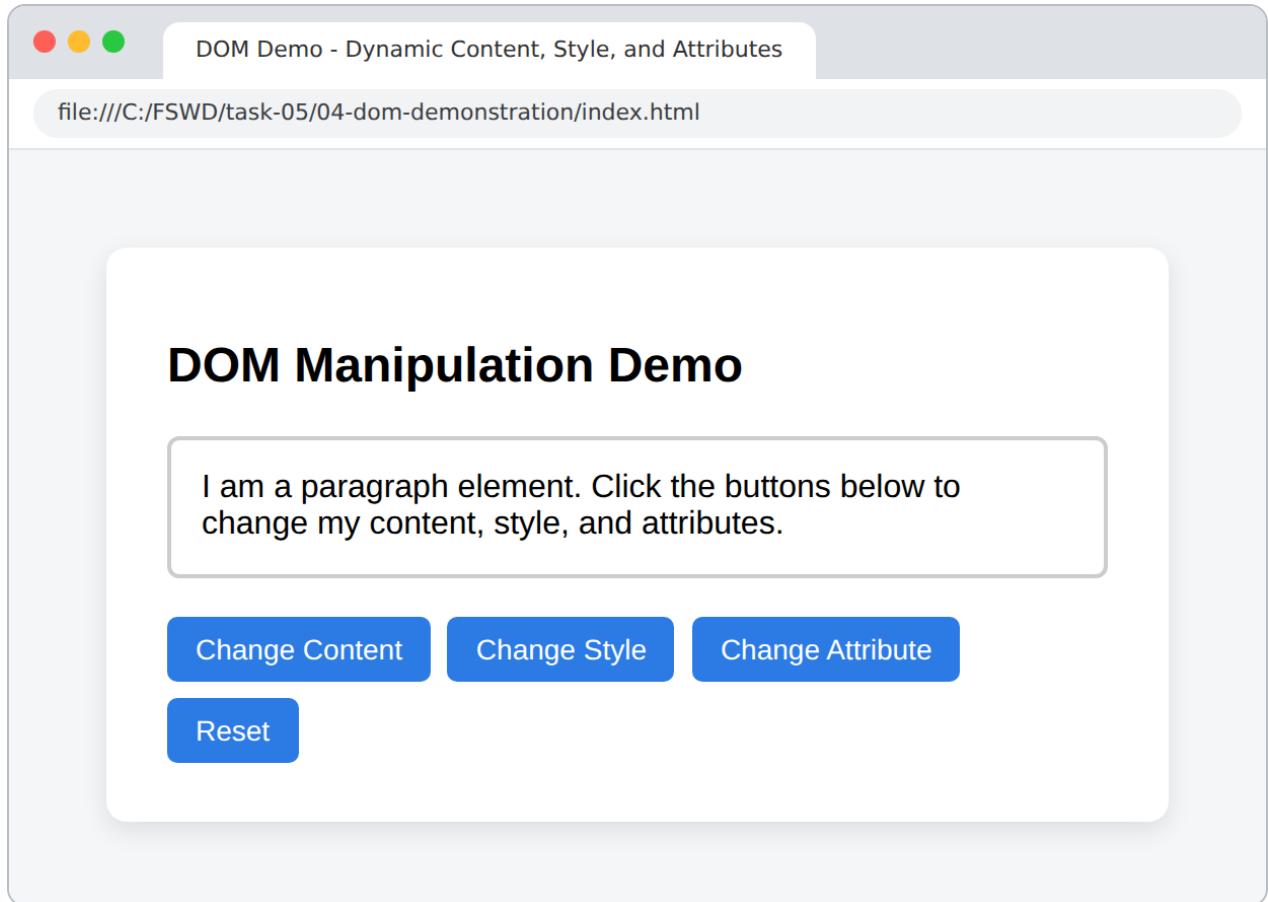
8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and each button was clicked individually as well as in combination, with the resulting element inspected directly in the DOM (outerHTML, computed attributes) after every click, in addition to a visual check and a developer-tools check.

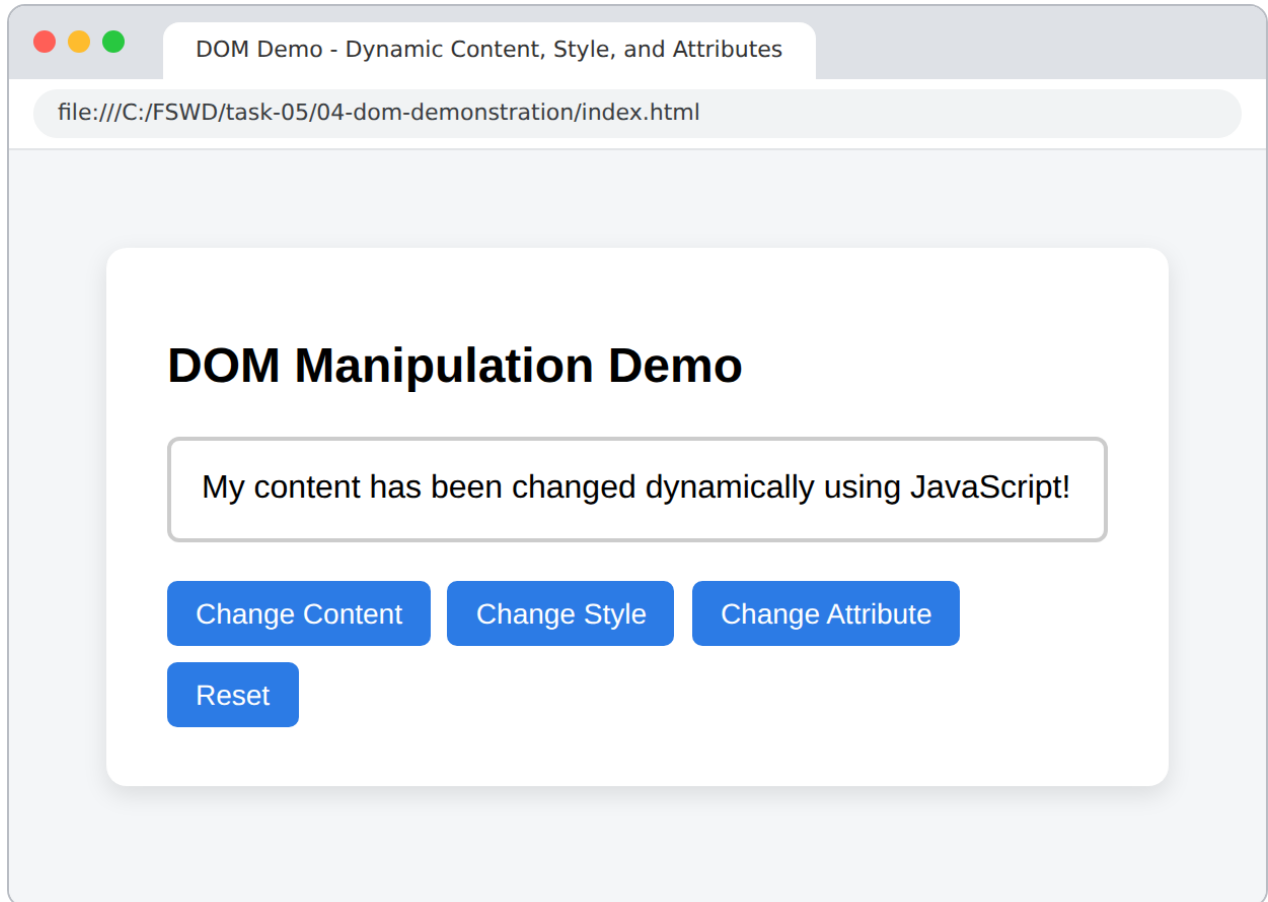
Test Case	Action / Input	Expected Result	Actual Result
TC-01	Open index.html in a browser	The original paragraph text is shown with no inline style, title "original tooltip"	Original content displayed — Pass
TC-02	Click "Change Content" only	The paragraph text changes; style and title remain untouched	Text changed, style/title unchanged — Pass
TC-03	Reload, then click "Change Style" only	The paragraph gets a green background, green bold text and a green border; text and title stay the same	Inline style applied, text/title unchanged — Pass
TC-04	Reload, then click "Change Attribute" only	The title attribute changes and a new data-modified="true" attribute appears on the element	title updated, data-modified="true" added — Pass
TC-05	Reload, then click Change Content, Change Style and Change Attribute in sequence	All three changes are visible together: new text, new inline style, new title and data-modified attribute	All three changes present simultaneously — Pass
TC-06	From the state in TC-05, click "Reset"	The element's outerHTML returns to exactly its original value: no style, no data-modified, original text and title	outerHTML matches the original exactly — Pass
TC-07	Make all three changes again, then refresh the page (no Reset click)	The DOM is rebuilt from the HTML file and the original outerHTML is restored	Original outerHTML restored after reload — Pass

9. OUTPUT

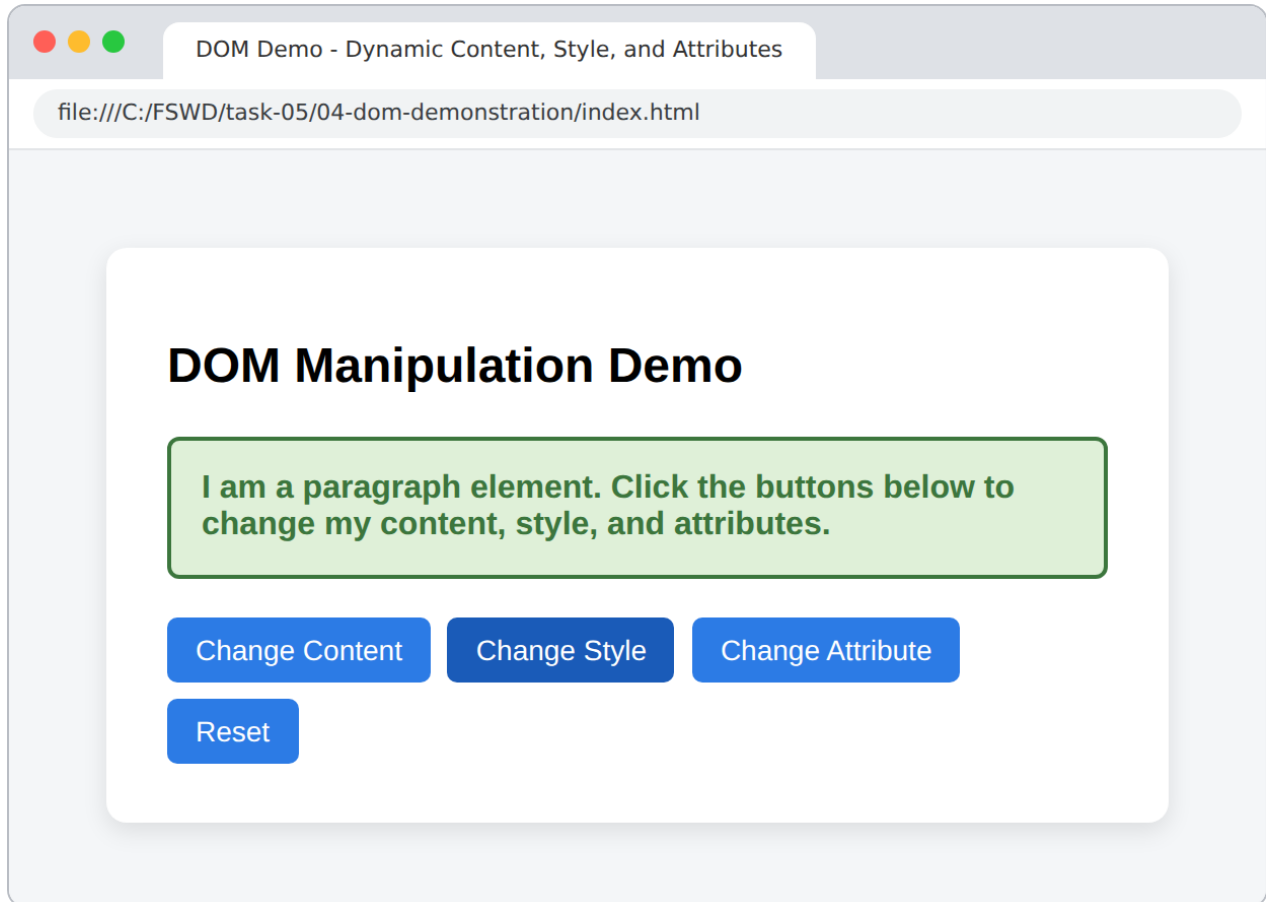
TC-01 – Original page before any button is clicked



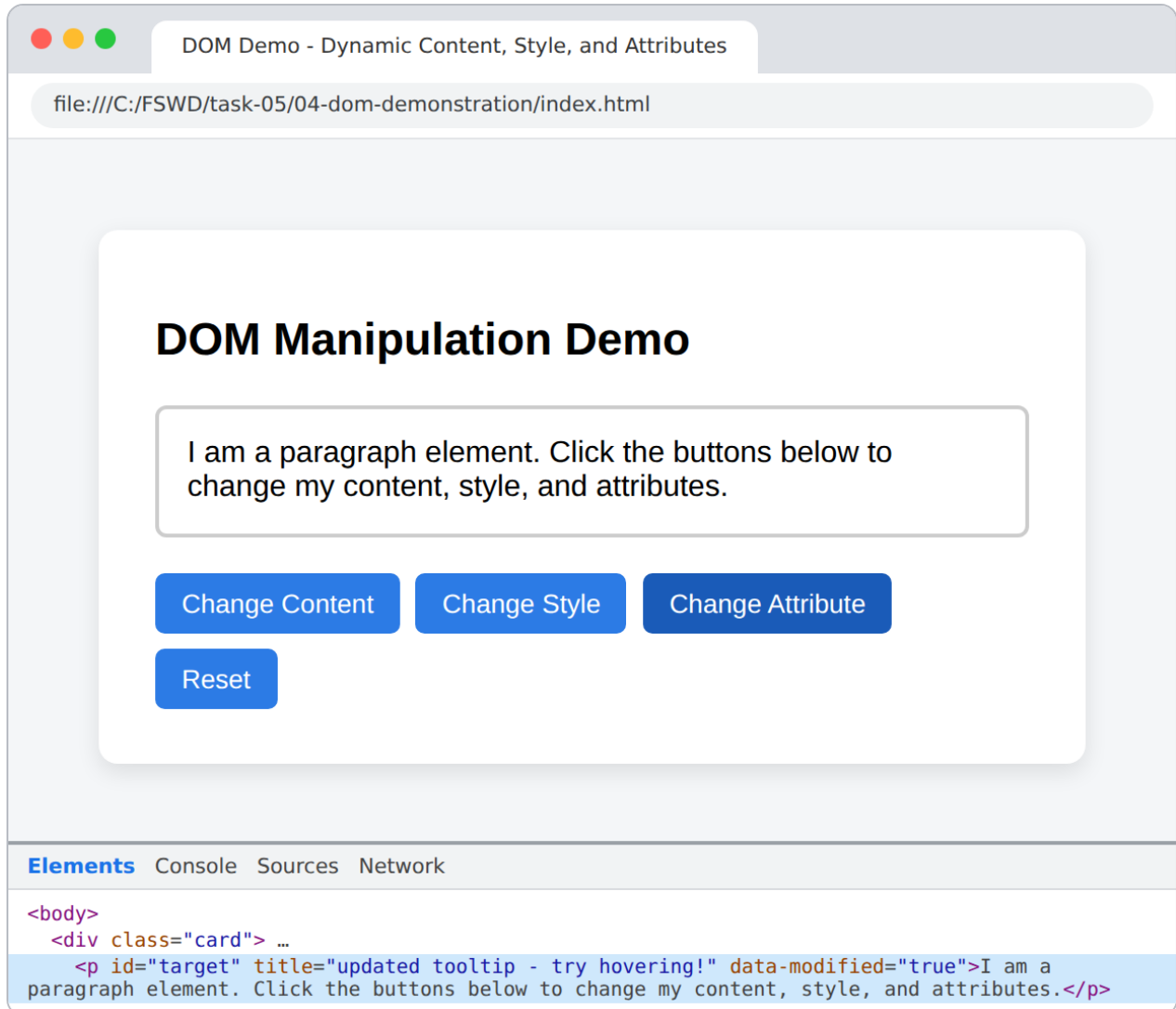
TC-02 – After "Change Content" only: text changed, style and title untouched



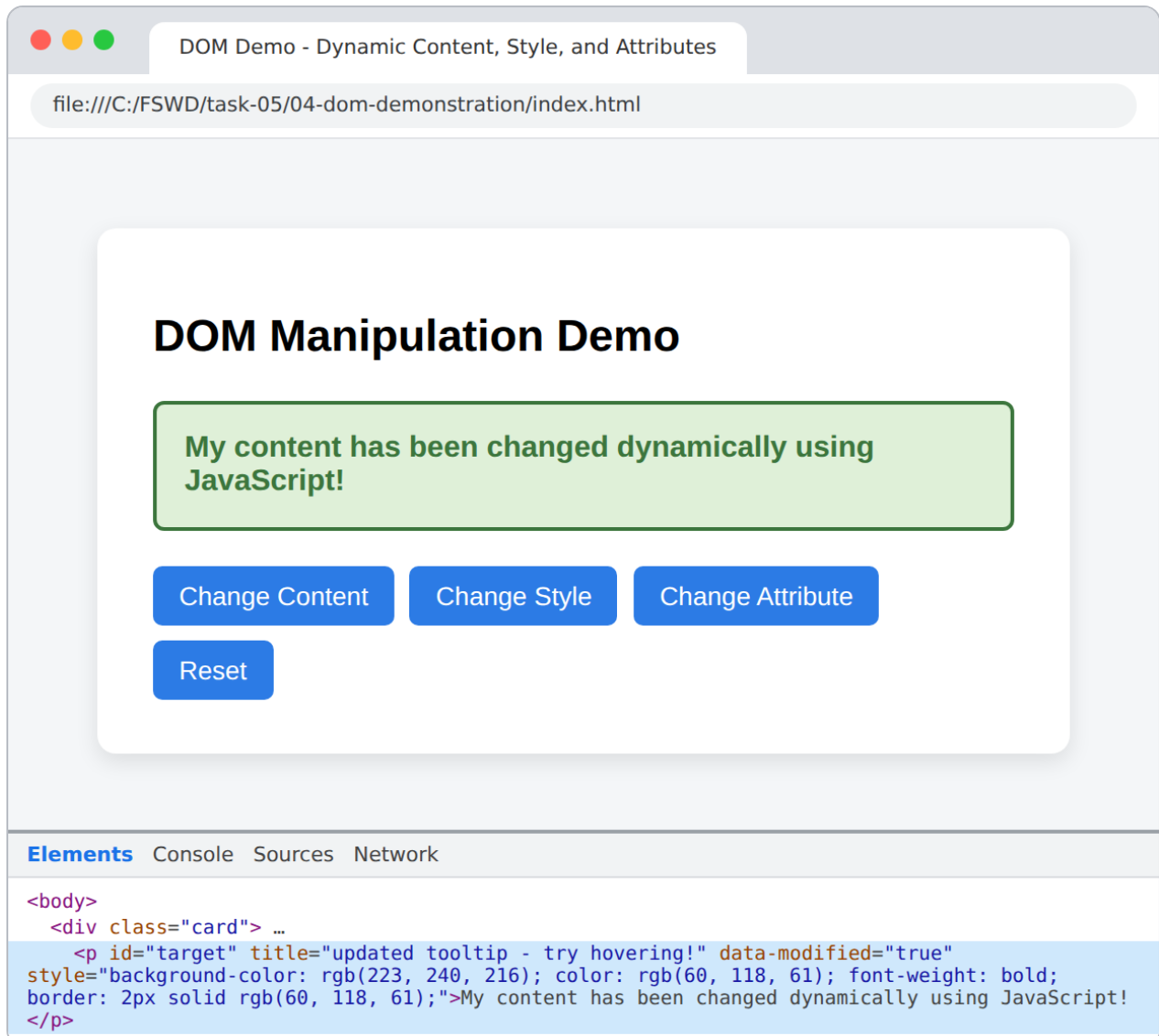
TC-03 – After "Change Style" only: background, text colour, weight and border applied



TC-04 – After "Change Attribute" only: title updated and data-modified attribute added (DevTools)



TC-05 – All three changes applied together: content, style and attributes (DevTools)



DOM Demo - Dynamic Content, Style, and Attributes

file:///C:/FSWD/task-05/04-dom-demonstration/index.html

DOM Manipulation Demo

My content has been changed dynamically using JavaScript!

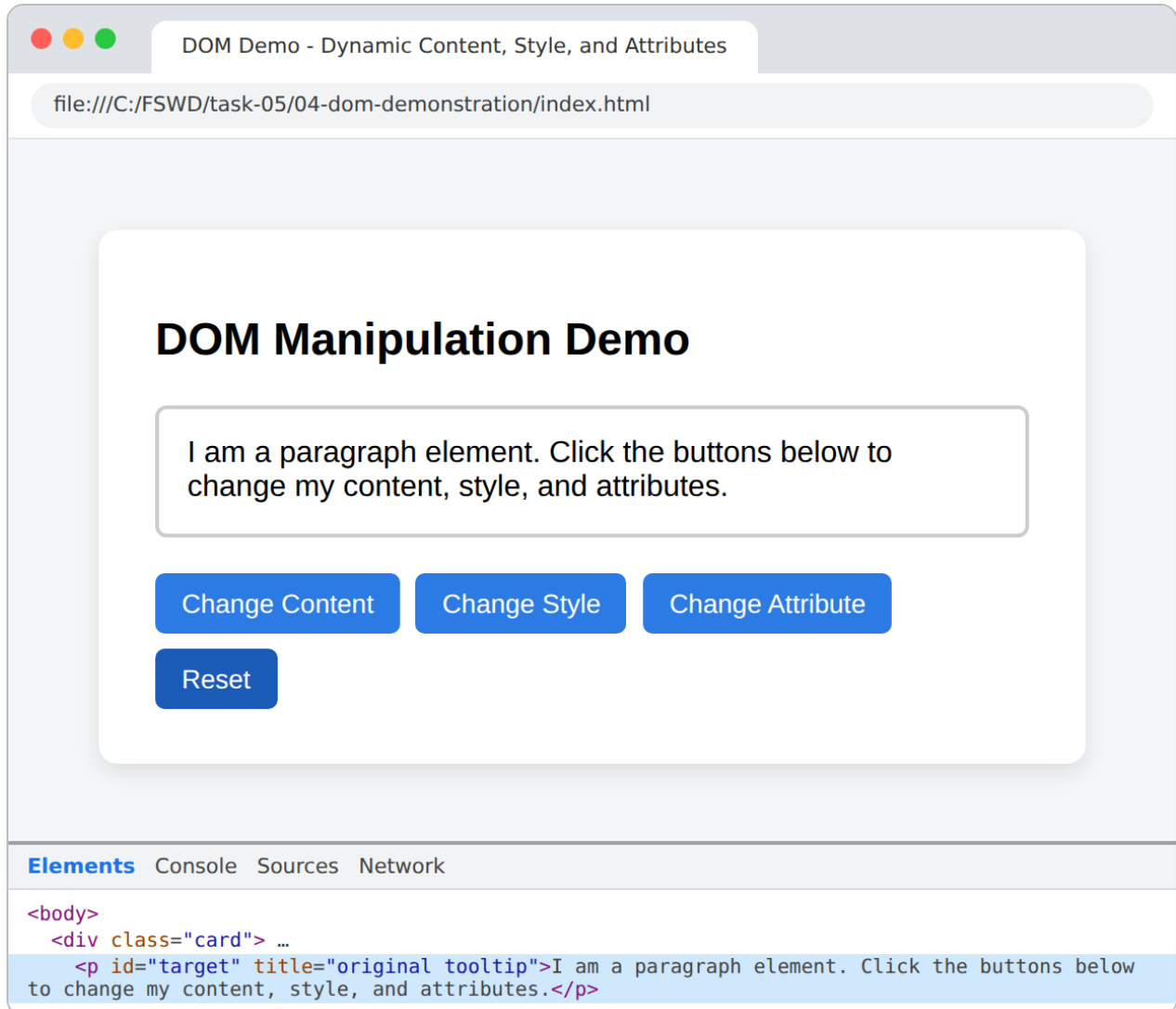
Change Content Change Style Change Attribute

Reset

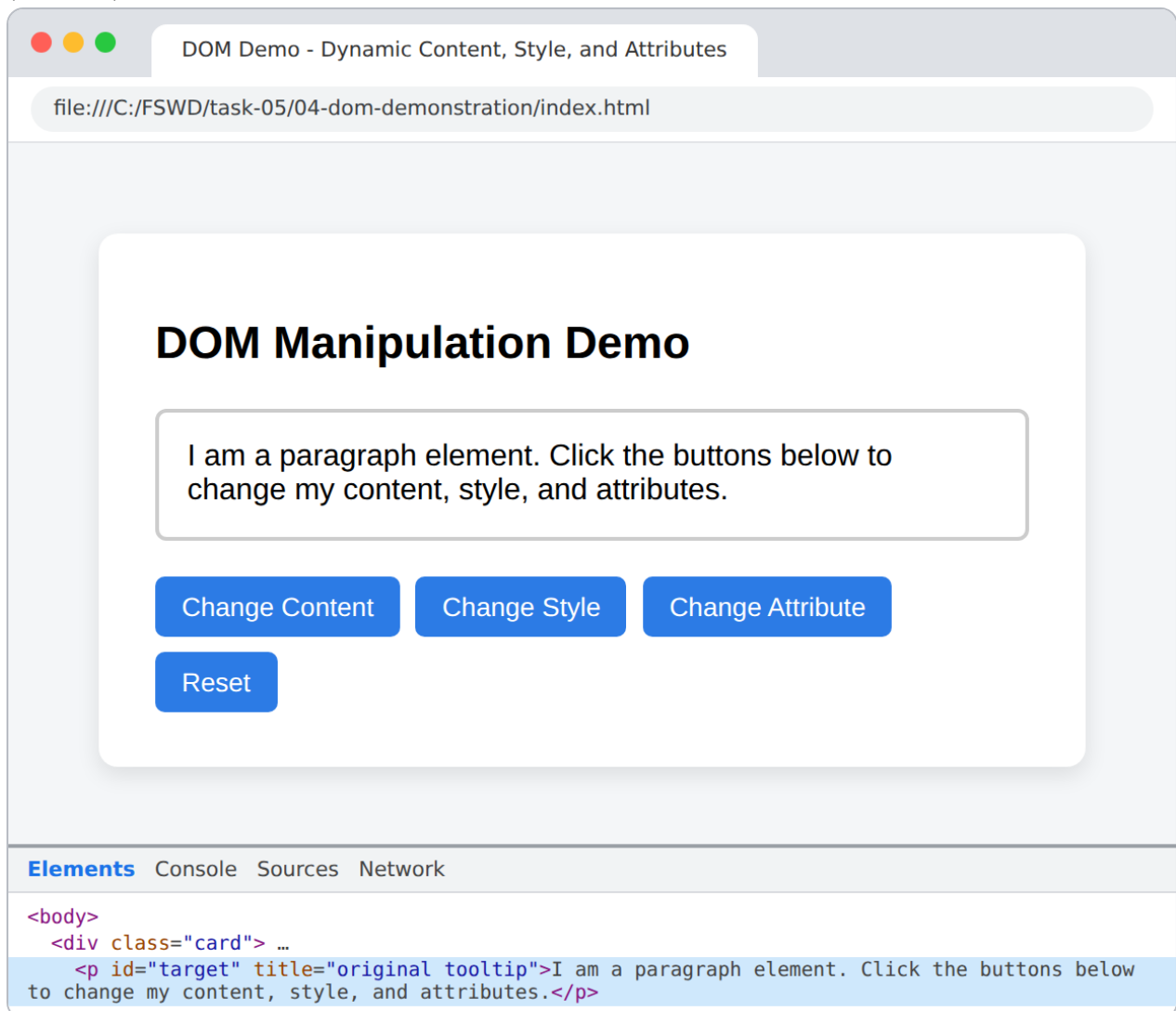
Elements Console Sources Network

```
<body>
  <div class="card"> ...
    <p id="target" title="updated tooltip - try hovering!" data-modified="true"
      style="background-color: rgb(223, 240, 216); color: rgb(60, 118, 61); font-weight: bold;
      border: 2px solid rgb(60, 118, 61);">My content has been changed dynamically using JavaScript!
    </p>
```

TC-06 – After clicking "Reset": element restored to its exact original outerHTML (DevTools)



TC-07 – After a page refresh (no Reset click): DOM rebuilt from the file, original state restored (DevTools)



10. RESULT

Thus, the Document Object Model was explained and demonstrated successfully. Three independent buttons changed the content, the style and the attributes of the same paragraph element using `textContent`, the `style` object and `setAttribute()` respectively, and the browser repainted the page immediately after each click without reloading it. A fourth "Reset" button, using `removeAttribute()` alongside `textContent` and `setAttribute()`, was verified to bring the element's `outerHTML` back to exactly its original value. The developer tools confirmed that the inline style and the new title and data-modified attributes were present on the live node in the DOM after each change, and a direct `outerHTML` comparison confirmed that refreshing the page — as opposed to clicking Reset — also restores the original content, because the DOM is rebuilt from the HTML file each time the page loads and every change made through JavaScript exists only in memory until then.